

GEX Dashboard Live

Real-time Greek Exposure (GEX) analysis and automated trading system for options markets, powered by a **Hybrid Attention-MLP** model.

Table of Contents

- [Overview](#)
 - [Project Structure](#)
 - [Services](#)
 - [Trading Bots](#)
 - [Hybrid Model](#)
 - [Backtesting](#)
 - [Discord Integration](#)
 - [Tools & Utilities](#)
 - [Data Directories](#)
 - [Configuration](#)
 - [Deployment](#)
-

Overview

This project provides a comprehensive suite for:

- **Real-time Greek exposure calculation** (Gamma, Vanna, Charm, Delta, DGEX, Vega, Vomma) across ODTE and Weekly expirations.
- **Automated trading** using a state-of-the-art **Hybrid Attention-MLP** neural network.
- **Initial Balance (IB) analysis** with Fibonacci extensions.
- **Discord notifications** for high-probability signals and trade management.
- **Backtesting framework** to validate strategies against historical data.

The system processes options chain data from TastyTrade, calculates complex Greek exposures, identifies key levels (magnet/accelerator), and executes trades based on ML predictions.

Project Structure

Gex-Dashboard-Live/

```
├─ services/                                # Background daemons (24/7)
│   ├── gex_daemon.py                      # Greek exposure data generator
│   ├── ib_service.py                     # Initial Balance chart generator
│   ├── servidor.py                       # HTTP API server (port 8609)
│   ├── fourier_service.py                # Fourier cycle analysis
│   ├── fourier_service_fast.py
│   └── calc_max_pain.py                  # Max Pain calculator
│
├─ neural/                                # Hybrid Neural Network (Attention-MLP)
│   ├── hybrid_model.py                  # Model Architecture (PyTorch)
│   ├── train_hybrid.py                  # Training script
│   └── feature_reduction.py             # PCA & feature engineering
│
├─ bots/                                  # Trading Logic & Execution
│   ├── tradingbot_wrapper.py            # Main entry point (loads neural model)
│   ├── tradingbot1.py                   # (Legacy) Multi-ticker confluence strategy
│   ├── tradingbot2.py                   # (Legacy) Level reversal strategy
│   ├── check_trades.py                  # Trade status monitor
│   ├── trades_wrapper/                  # Active & closed trades (JSON)
│   └── trades_live/                     # Legacy trades
│
├─ models/                                # Trained models
│   ├── trading_hybrid.pt                 # PyTorch model weights
│   └── hybrid_normalizer.npz            # Feature scalers
│
├─ discord_app/                           # Discord bot integration
│   ├── main.py                          # Discord bot entry point
│   ├── bot.py                           # Bot commands handler
│   └── discord_send_plots.py            # Plot scheduler
│
├─ backtest/                              # Backtesting scripts
│   ├── backtest_hybrid.py               # ML-based backtesting
│   ├── ib_backtest.py                   # IB data fetcher
│   └── plots/                           # Backtest visualizations
│
├─ scripts/                               # Scripts
│   ├── backfill_json_greeks.py          # Greek exposure data
│   └── fix_dates.py                     # Fix dates in JSON files
│
├─ tools/                                 # CLI utilities
```

```

├─ cli-app.py          # Manual data plotting
├─ data_plotting.py    # Data plotting tools
├─ watchdog_notify.py  # Watchdog notifications
|
├─ web/                # Web dashboard
|   └─ templates/
|       ├─ index.html      # Main HTML page
|       ├─ index_modular.html # Modular version (loads JS modules)
|       ├─ script.js       # Legacy monolithic script (backup)
|       ├─ styles.css      # Main CSS (imports modular files)
|       ├─ css/            # Modular CSS structure
|           ├─ base.css    # Variables, themes, typography
|           ├─ layout.css  # Main layout, tabs
|           ├─ components.css # Buttons, inputs, market monitor
|           ├─ charts.css  # Chart panels, heatmap, regime
|           ├─ login.css   # Login screen
|           └─ modes.css   # Zen mode, detached modes
|       └─ js/             # Modular JavaScript (16 modules)
|           ├─ state.js    # Global state variables
|           ├─ api.js      # Data fetching functions
|           ├─ ui.js       # UI helpers, time, themes
|           ├─ tabs.js     # Tab management
|           ├─ dragdrop.js # Drag and drop for panels
|           ├─ charts.js   # Chart panel dispatcher
|           ├─ heatmap.js  # Greek exposure heatmaps
|           ├─ fourier.js  # Fourier analysis panels
|           ├─ ib.js       # Initial Balance panels
|           ├─ handlers.js # Form handlers
|           ├─ history.js  # Time machine controls
|           ├─ layout.js   # Save/load layouts
|           ├─ refresh.js  # Dashboard refresh logic
|           ├─ windows.js  # Detached windows
|           ├─ auth.js     # Authentication
|           └─ init.js     # App initialization
|
├─ modules/            # Shared modules
|   ├─ stats.py         # Statistical functions
|   ├─ tasty_handler.py # TastyTrade API client
|   ├─ ticker_dwn.py    # Ticker data downloader
|   └─ utils.py         # Common utilities

```

```
└─ systemd/                # Systemd service files
   └─ *.service
   |
└─ .env                    # Environment variables
└─ requirements.txt        # Python dependencies
└─ README.md
```

Services

Background daemons that run continuously to generate data.

gex_daemon.py

Generates Greek exposure data files every minute during market hours.

```
python services/gex_daemon.py
```

Output: JSON files in `/home/.../json_data/`

- `{TICKER}_0dte_ExposureData_{DATE}_{TIME}.json`

Data includes:

- Gamma, Vanna, Charm, Delta, DGEX exposures
- Spot price
- Strike-level breakdown

ib_service.py

Generates Initial Balance charts using TastyTrade candle data.

```
python services/ib_service.py
```

Output:

- JSON: `/home/.../ib_charts/ib_data_{TICKER}_{DATE}.json`
- PNG: `/home/.../ib_charts/ib_chart_{TICKER}_{DATE}.png`

Features:

- IB range detection (9:30-10:30 NYSE)
 - Fibonacci extensions from IB range
 - Volume profile calculation
-

servidor.py

HTTP API server exposing Greek data for the web dashboard, with authentication.

```
python services/servidor.py
```

Port: 8609

Public Endpoints (no auth):

- `POST /login` – Login with email/password, returns Bearer token
- `GET /verify_token` – Check if a Bearer token is still valid
- `GET /` – Serve web dashboard
- `GET /generate_bit` – Quantum random bit (for Unity)

Protected Endpoints (require auth):

- `GET /get_latest?ticker=SPX&exp=0dte` – Latest Greek data from RAM cache
 - `GET /get_history?ticker=SPX&exp=0dte&time=0930` – Historical snapshot by time
 - `GET /list_files?ticker=SPX&exp=0dte&date=20260214` – List available JSON files
 - `POST /get_batch` – Batch fetch multiple tickers (used by dashboard refresh)
-

Authentication & API Keys

The server uses a hybrid authentication system with two mechanisms:

1. Bearer Tokens (Web Users)

Tokens are **UUID4 strings** generated server-side when a user logs in via `POST /login`. They are stored in `SESSIONS` (server RAM) and sent back to the client.

How it works:

1. User submits email + password to `POST /login`
2. Server validates against `USERS` dict → generates `uuid.uuid4()` token

3. Token is stored in `SESSIONS = { token: {"email": ..., "role": ...} }`

4. Client stores token in `sessionStorage` and sends it with every request as:

```
Authorization: Bearer <token>
```

5. Server checks `SESSIONS[token]` on each protected request

Single-session enforcement:

- When a non-admin user logs in, any **previous token for that email is deleted**
- The old session's next API request returns `401 Unauthorized` → auto-logout
- **Admin users** are exempt — they can have multiple active sessions

Example login request:

```
curl -X POST http://gex-dashboard.hopto.org:8609/login \
-H "Content-Type: application/json" \
-d '{"email": "user1@flowgreeks.com", "password": "FlowGreeksPlottingUs

# Response: {"status": "ok", "token": "a1b2c3d4-...", "role": "USER"}
```

Example authenticated request:

```
curl http://gex-dashboard.hopto.org:8609/get_latest?ticker=SPX&exp=0dte \
-H "Authorization: Bearer a1b2c3d4-..."
```

2. API Keys (Scripts & Bots)

API keys are **static strings** defined in `servidor.py` for automated scripts. Each key allows **one concurrent request** (rate limited via threading locks).

Adding API keys — Edit `API_KEYS` in `services/servidor.py`:

```
API_KEYS = {
    "gex_bot_2026_xyz": {"role": "BOT", "owner": "trading_bot"},
    "client_abc_secret": {"role": "USER", "owner": "client_abc"},
}
# IMPORTANT: Regenerate locks after changing keys
API_KEY_LOCKS = {k: threading.Lock() for k in API_KEYS}
```

Using an API key in a script:

```
import requests

headers = {"X-API-Key": "gex_bot_2026_xyz"}
resp = requests.get("http://your-server:8609/get_latest?ticker=SPX&exp=0c
data = resp.json()

# Or with curl:
curl http://your-server:8609/get_latest?ticker=SPX&exp=0dte \
-H "X-API-Key: gex_bot_2026_xyz"
```

Concurrency: If two requests arrive simultaneously with the same API key, the second gets `429 Too Many Requests`. Use different keys for different scripts if you need parallel access.

Auth Summary

Method	Header	Who	Concurrency
Bearer Token	<code>Authorization: Bearer <uuid></code>	Web users	1 session/email (admin exempt)
API Key	<code>X-API-Key: <key></code>	Scripts/bots	1 request/key at a time
None	—	—	<code>401 Unauthorized</code>

Managing Users

Web users are defined in `services/servidor.py`:

```
USERS = {
    "admin@flowgreeks.com": {"pass": "admin123", "role": "ADMIN"},
    "flowgreeks@email.com": {"pass": "FlowGreeksPlotting", "role": "USER"},
    # Add more users here
}
```

Note: Tokens live only in server RAM (`SESSIONS` dict). Restarting the server invalidates all active sessions — users must log in again.

Authentication & API Keys

The server uses a hybrid authentication system with two mechanisms:

1. Bearer Tokens (Web Users)

Tokens are **UUID4 strings** generated server-side when a user logs in via `POST /login`. They are stored in `SESSIONS` (server RAM) and sent back to the client.

How it works:

1. User submits email + password to `POST /login`
2. Server validates against `USERS` dict → generates `uuid.uuid4()` token
3. Token is stored in `SESSIONS = { token: {"email": ..., "role": ...} }`
4. Client stores token in `sessionStorage` and sends it with every request as:

```
Authorization: Bearer <token>
```

5. Server checks `SESSIONS[token]` on each protected request

Single-session enforcement:

- When a non-admin user logs in, any **previous token for that email is deleted**
- The old session's next API request returns `401 Unauthorized` → auto-logout
- **Admin users** are exempt – they can have multiple active sessions

Example login request:

```
curl -X POST http://gex-dashboard.hopto.org:8609/login \
-H "Content-Type: application/json" \
-d '{"email": "user1@flowgreeks.com", "password": "FlowGreeksPlottingUs

# Response: {"status": "ok", "token": "a1b2c3d4-...", "role": "USER"}
```

Example authenticated request:

```
curl http://gex-dashboard.hopto.org:8609/get_latest?ticker=SPX&exp=0dte \
-H "Authorization: Bearer a1b2c3d4-..."
```

2. API Keys (Scripts & Bots)

API keys are **static strings** defined in `servidor.py` for automated scripts. Each key allows **one concurrent request** (rate limited via threading locks).

Adding API keys – Edit `API_KEYS` in `services/servidor.py`:

```

API_KEYS = {
    "gex_bot_2026_xyz": {"role": "BOT", "owner": "trading_bot"},
    "client_abc_secret": {"role": "USER", "owner": "client_abc"},
}
# IMPORTANT: Regenerate locks after changing keys
API_KEY_LOCKS = {k: threading.Lock() for k in API_KEYS}

```

Using an API key in a script:

```

import requests

headers = {"X-API-Key": "gex_bot_2026_xyz"}
resp = requests.get("http://your-server:8609/get_latest?ticker=SPX&exp=0c
data = resp.json()

# Or with curl:
curl http://your-server:8609/get_latest?ticker=SPX&exp=0dte \
-H "X-API-Key: gex_bot_2026_xyz"

```

Concurrency: If two requests arrive simultaneously with the same API key, the second gets **429 Too Many Requests**. Use different keys for different scripts if you need parallel access.

Auth Summary

Method	Header	Who	Concurrency
Bearer Token	<code>Authorization: Bearer <uuid></code>	Web users	1 session/email (admin exempt)
API Key	<code>X-API-Key: <key></code>	Scripts/bots	1 request/key at a time
None	—	—	401 Unauthorized

Managing Users

Web users are defined in `services/servidor.py`:

```

USERS = {
    "admin@flowgreeks.com": {"pass": "admin123", "role": "ADMIN"},
    "flowgreeks@email.com": {"pass": "FlowGreeksPlotting", "role": "USER"}
}

```

```
# Add more users here
}
```

Note: Tokens live only in server RAM (`SESSIONS` dict). Restarting the server invalidates all active sessions – users must log in again.

fourier_service.py / fourier_service_fast.py

Performs Fourier analysis to detect market cycles.

```
python services/fourier_service.py          # Standard
python services/fourier_service_fast.py     # Faster updates
```

Trading Bots

Automated trading bots that execute based on Greek signals.

tradingbot1.py – Multi-Ticker Confluence Strategy

Requires signal alignment across SPX, SPY, and QQQ before entering.

```
python bots/tradingbot1.py
```

Strategy Logic:

1. **Positive Gamma:** Mean-reversion at support/resistance
2. **Negative Gamma:** Momentum following with Vanna/Charm signals
3. **Min Vanna Magnet:** Trades toward untouched Min Vanna levels

Trade Management:

Parameter	Value
Stop Loss	0.3%
Profit Target	1.2%
Emergency Stop	0.5%
Min Holding Time	25 minutes
Cooldown After Exit	10 minutes

Parameter	Value
Force Exit	15:55 NYC

State Persistence:

- Active trade saved to `bots/state_bot1.json`
- Completed trades saved to `bots/trades_live/`
- Logs written to `bots/tradingbot1.log`

Discord Notifications:

- Trade open with detailed reasoning (gamma regime, vanna signal, levels)
- Trade close with P&L breakdown and duration

tradingbot2.py - Level Reversal Strategy

Trades individual tickers based on level tests and momentum breakouts.

```
python bots/tradingbot2.py
```

Strategy Types:

Type	Trigger	Target
REVERSAL	Price at support/resistance	Untouched Min Vanna or opposite level
MOMENTUM	Breakout of key level in negative gamma	DGEX accelerator level

Trade Management:

Parameter	Value
Stop Loss	0.3% (fixed)
Breakeven Trigger	+0.25%
Trailing Step	0.2%
Cooldown	30 minutes
Force Exit	15:55 NYC

State Persistence:

- Active trade: `bots/state_bot2.json`

- Completed trades: `bots/trades_live2/`
 - Logs: `bots/tradingbot2.log`
-

check_trades.py - Trade Monitor

Quick status check for active trades across both bots.

```
python bots/check_trades.py
```

Features:

- Shows active trade details (entry, duration, unrealized P&L)
- Fetches current spot from Greek data files
- Detects and explains contradictory positions between bots
- Displays entry reasoning and Greek signals

Example Output:

```
=====
TRADINGBOT1:  LONG SPX
=====

Entry Price:   $6050.25
Duration:      1h 30m
 Unrealized:   +5.75 pts (+0.09%) = $57.50

WHY THIS TRADE?
Reason: Pos Gamma Bounce at 6045.00
Gamma Regime: LONG
Min Vanna: $6065.00 - UNTOUCHED (magnet)
```

Backtesting

Scripts to test strategies on historical data.

backtest_hybrid.py (PRIMARY)

Backtests the **Hybrid Attention-MLP** model on historical 1-minute data, simulating realistic execution with fees and slippage.

```
python neural/backtest_hybrid.py --data training_data/backtest_trades.csv
```

Key Features:

- Uses actual 1-minute OHLC candles (IB Data) for accurate execution simulation
- Comprehensive metrics: Sharpe, Profit Factor, Max Drawdown
- Detailed logs: `logs/backtest_hybrid.log`
- Feature Importance analysis for winning vs losing trades

backtest.py

Backtests the multi-ticker confluence strategy.

```
python backtest/backtest.py
```

Requires: Historical IB data in `ib_backtest/`

backtest2.py

Backtests the level reversal strategy.

```
python backtest/backtest2.py
```

plot_backtest.py - Trade Visualizer

Generates candlestick charts with trade markers and levels.

```
# Basic usage (backtest trades)
python backtest/plot_backtest.py
```

```
# Specify source
```

```
python backtest/plot_backtest.py --source backtest # Backtest trades
python backtest/plot_backtest.py --source bot1    # TradingBot1 live tr
python backtest/plot_backtest.py --source bot2    # TradingBot2 live tr
python backtest/plot_backtest.py --source all     # All sources combine
```

```
# Specific date
```

```
python backtest/plot_backtest.py --source bot1 --date 20260127
```

```
# Short flags
python backtest/plot_backtest.py -s all -d 20260127
```

Arguments:

Flag	Options	Default	Description
<code>--source , -s</code>	<code>backtest , bot1 , bot2 , all</code>	<code>backtest</code>	Trade data source
<code>--date , -d</code>	<code>YYYYMMDD</code>	All dates	Specific date to plot

Output: PNG files in `backtest/plots/`

- `candles_{TICKER}_{DATE}_{SOURCE}.png` - Candlestick with trades
- `volume_profile_{TICKER}_{DATE}.png` - Volume profile chart

Chart Features:

- 1-minute candlesticks
- Entry/exit markers (O = Open, C = Close)
- Support/resistance levels
- IB high/low
- Min Vanna magnet
- Volume profile (VPOC, VAH, VAL)

ib_backtest.py

Fetches and stores historical IB data for backtesting.

```
python backtest/ib_backtest.py
```

Output: JSON files in `ib_backtest/`

Discord Integration

main.py

Entry point for the Discord bot. Runs both the command handler and plot scheduler.

```
python discord_app/main.py
```

Features:

- Scheduled plot posting to channels
 - Interactive command responses
 - Systemd notify integration
-

bot.py

Discord bot command handler.

Commands:

```
#load SPX 0dte gamma      - Generate and post specific plot
#load QQQ weekly vanna    - Weekly vanna exposure for QQQ
```

Usage:

```
#load <TICKER> <EXPIRATION> <GREEK>
```

TICKER: SPX, SPY, QQQ, etc.

EXPIRATION: 0dte, 1dte, weekly, opex, monthly, all

GREEK: delta, gamma, vanna, charm

discord_send_plots.py

Handles scheduled plot posting and Discord API interactions.

Scheduled Posts:

- Greek exposure plots at configurable intervals
 - IB charts after market open
 - Summary reports at market close
-

Tools & Utilities

cli-app.py

Manual data plotting from command line.

```
python tools/cli-app.py SPX 0dte gamma
python tools/cli-app.py QQQ weekly vanna
```

Arguments:

```
python tools/cli-app.py <TICKER> <EXPIRATION> [GREEK]
```

TICKER: SPX, SPY, QQQ, etc.

EXPIRATION: 0dte, 1dte, weekly, opex, monthly, all

GREEK: delta, gamma, vanna, charm (optional)

Data Directories

The system uses several data directories (configured with absolute paths):

Directory	Purpose	Generated By
json_data/	Greek exposure JSON files	gex_daemon.py
ib_charts/	Real-time IB data & charts	ib_service.py
ib_backtest/	Historical IB data	ib_backtest.py
bots/trades_live/	Bot1 completed trades	tradingbot1.py
bots/trades_live2/	Bot2 completed trades	tradingbot2.py
backtest/trades/	Backtest trade outputs	backtest.py
backtest/plots/	Trade visualization PNGs	plot_backtest.py

Configuration

Environment Variables

Create a `.env` file in the project root:

```
# Discord
DISCORD_WEBHOOK_URL=https://discord.com/api/webhooks/...
DISCORD_BOT_TOKEN=your_bot_token_here

# TastyTrade API
```

```
TT_USERNAME=your_tastytrade_username
TT_PASSWORD=your_tastytrade_password
```

Trading Parameters

Trading parameters are configured as constants at the top of each bot file:

tradingbot1.py:

```
STOP_LOSS_PCT = 0.003          # 0.3%
PROFIT_TARGET_PCT = 0.012      # 1.2%
EMERGENCY_STOP_LOSS_PCT = 0.005 # 0.5%
MIN_HOLDING_TIME_MINUTES = 25
COOLDOWN_AFTER_EXIT_MINUTES = 10
MIN_CONFIDENCE_THRESHOLD = 0.55
```

tradingbot2.py:

```
STOP_LOSS_FIXED = 0.003        # 0.3%
BREAKEVEN_TRIGGER = 0.0025     # 0.25%
TRAILING_STEP = 0.002          # 0.2%
COOLDOWN_MINUTES = 30
MIN_RISK_REWARD = 1.2
```

Deployment

1. Clone and Setup

```
git clone https://github.com/kripta-studios/gex-dashboard-live.git
cd gex-dashboard-live
pip install -r requirements.txt
cp .env.example .env # Edit with your credentials
```

2. Deploy to Server

```
rsync -avz --exclude='.git' --exclude='__pycache__' --exclude='*.pyc' \
./ root@your-server:/home/Option-Greeks-Plotting-Discord-Bot/
```

3. Install Systemd Services

```
# Copy service files
sudo cp systemd/*.service /etc/systemd/system/

# Reload systemd
sudo systemctl daemon-reload

# Enable services to start on boot
sudo systemctl enable gex_daemon ib discord-bot tradingbot1 tradingbot2

# Start services
sudo systemctl start gex_daemon ib discord-bot tradingbot1 tradingbot2
```

4. Monitor Services

```
# Check status
sudo systemctl status tradingbot1

# View logs
sudo journalctl -u tradingbot1 -f

# Restart a service
sudo systemctl restart tradingbot1
```

4. Monitor Services

```
# Check status
sudo systemctl status tradingbot1

# View logs
sudo journalctl -u tradingbot1 -f

# Restart a service
sudo systemctl restart tradingbot1
```

Development

Code Style

- Python 3.9+
- Type hints encouraged
- Docstrings for all functions
- Constants in UPPER_CASE

Adding a New Strategy

1. Create `bots/tradingbot3.py` based on existing templates
2. Implement `generate_signal()` with your logic
3. Create `systemd/tradingbot3.service`
4. Update this README

Running Tests

```
# Verify all Python files compile
find . -name "*.py" -exec python3 -m py_compile {} \;

# Run a specific backtest
python backtest/backtest.py
```

Project Dependencies

Key dependencies (see `requirements.txt`):

- `requests` - HTTP client
- `numpy` - Numerical operations
- `pandas` - Data manipulation
- `matplotlib` - Chart generation
- `discord.py` - Discord bot
- `python-dotenv` - Environment variables
- `pytz` / `zoneinfo` - Timezone handling

License

MIT License - See [LICENSE](#) for details.

Contributing

1. Fork the repository
2. Create a feature branch(`git checkout -b feature/new-strategy`)
3. Commit changes(`git commit -am 'Add new strategy'`)
4. Push to branch(`git push origin feature/new-strategy`)
5. Open a Pull Request

For major changes, please open an issue first to discuss.